

Troubleshooting AWS

Erros HTTP, Exceções SDK e Padrões de Retry

AWS Certified Developer Associate

Instrutor: Marlon Anesi

Códigos HTTP Críticos no Exame

Código	Nome	Significado AWS e Ação
200	OK	Sucesso. Resposta padrão de operações bem-sucedidas.
400	Bad Request	Parâmetros inválidos ou malformados. Corrija a requisição antes de reenviar.
403	Forbidden (AccessDenied)	IAM negou a requisição. Revise as policies — NAO retente, erro de permissão.
404	Not Found	Recurso não existe. Ex: chave inexistente no DynamoDB ou bucket S3.
409	Conflict	Conflito de estado. Ex: ConditionalCheckFailedException no DynamoDB.
429	Too Many Requests	Throttling — limite de taxa atingido. Retente com exponential backoff + jitter.
500	Internal Server Error	Falha no servidor AWS. Retente com backoff. Pode ser falha transitória.
503	Service Unavailable	Serviço temporariamente indisponível. Retente com backoff exponencial.

Exceções AWS SDK Mais Cobradas na Prova

Exceções do SDK são subclasses de ClientError (Python boto3) ou AWSException. Cada uma indica uma causa específica.

ProvisionedThroughputExceededException

DynamoDB

Throttling de leitura/escrita. Use exponential backoff ou aumente RCU/WCU.

RequestExpired

Geral

Timestamp da requisição muito defasado (>15 min). Sincronize o relógio do host.

AccessDeniedException

IAM

Permissão IAM negada. Revise policies — nunca retente sem corrigir permissão.

ThrottlingException

Geral

Limite de API atingido. Retente com backoff + jitter. Comum em EC2, IAM, STS.

ResourceNotFoundException

Vários

Recurso referenciado não existe. Verifique nome do recurso e região correta.

ServiceUnavailableException

Vários

Serviço temporariamente indisponível. Retente com backoff exponencial.

Regra de ouro: 4xx = erro do cliente (não retente sem corrigir). 5xx = erro do servidor (retente com backoff). 429 = throttling (retente).

Throttling e Padrão de Retry com Backoff

Código Python — Exponential Backoff com Jitter:

```
import time, random
def call_with_retry(fn, retries=5):
    for attempt in range(retries):
        try:
            return fn()
        except ThrottlingException:
            wait = (2 ** attempt) +
                  random.random()
            time.sleep(wait)
    raise Exception("Max retries")
```

DynamoDB Throttle

Aumentar RCU/WCU ou usar On-Demand. Backoff no SDK já vem configurado.

Lambda Throttle (429)

Concorrência esgotada. Aumente reserved concurrency ou use SQS como buffer.

API Gateway 429

Usage Plan atingido. Aumente quota ou implemente fila com SQS.

S3 Rate Limit (5xx)

Prefixos de chave diversificados reduzem hotspots. S3 escala por prefixo.

O AWS SDK (boto3, Java SDK) já implementa backoff automático para 5xx e 429. Configure `max_attempts` e `mode="adaptive"` no Config.

Pontos-chave para o Exame

1

Códigos HTTP Críticos

- 403 = IAM negou (AccessDenied) — nunca retente sem corrigir a policy
- 429 = Throttling (TooManyRequests) — retente com exponential backoff + jitter
- 5xx (500, 503) = falha do servidor — retente com backoff (pode ser transitório)

2

Exceções SDK Mais Cobradas

- ProvisionedThroughputExceededException = DynamoDB sem capacidade (RCU/WCU)
- ThrottlingException = limite de API atingido — backoff + jitter
- AccessDeniedException = IAM policy incorreta — não retente, corrija permissão

3

Padrão de Retry com Backoff

- Exponential backoff: tempo de espera dobra a cada tentativa (1s, 2s, 4s, 8s...)
- Jitter: adicionar randomização para evitar thundering herd problem
- AWS SDK boto3: use `Config(retries={"max_attempts":5, "mode":"adaptive"})`

**LEMBRE-SE: 429 = throttling (retente com backoff+jitter). 403 = IAM negou (nao retente). 5xx = servidor (retente).
ProvisionedThroughputExceededException = DynamoDB!**